# Plugin Architectures with Rust

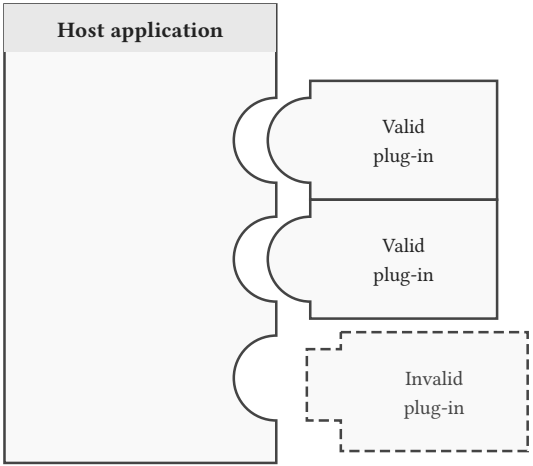
## an Analysis of Obstacles and Prospects

20.05.2026


# Contents

# 1 Introduction

# Plugin Systems




- **Extensibility**: The ability to add new functionality post-deployment.
- **Modularity**: Plugins are isolated units that interact with the host through a *well-defined interface*.
- **Dynamic Loading**: Plugins are typically loaded at runtime rather than linked statically at compile time.

# Criteria

**Performance Overhead**

Resulting from the required overhead, how slow is the execution of the plugin compared to the rust native solution.

This is measured by using a average data fusion with 1.000.000 data points and expressed with **percentages**.

**Development Complexity**

- Learning curve of the implementation
- Quality of the Documentation and examples
- Required amount of work and consequences for setup, configurations and integration

This is expressed from **--** to **++**.

# Criteria (ii)

**(Language) Limitations**

How much does this approach limit the usage of the rust languages features and ecosystem.

This is expressed from **-- to 0**.

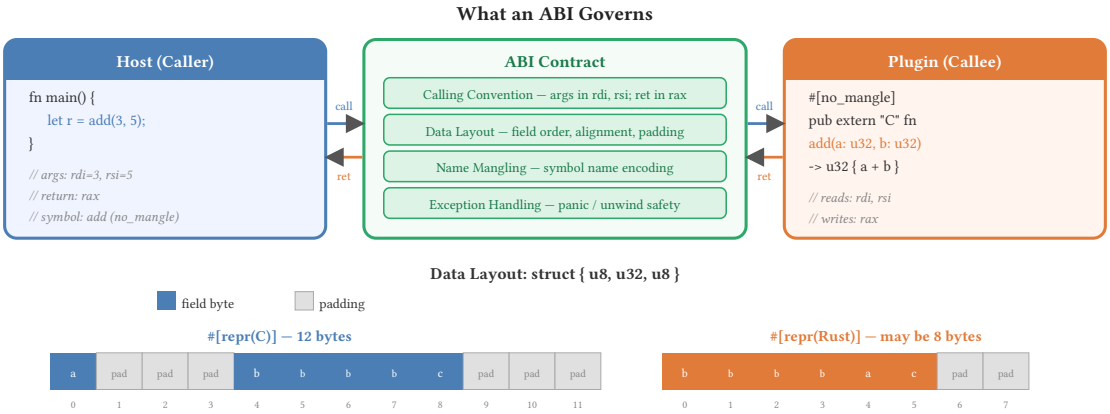
**Interoperability**

How well can this approach work with other libraries, tools and especially languages.

This is expressed from **0 to ++**.

# 2 ABI

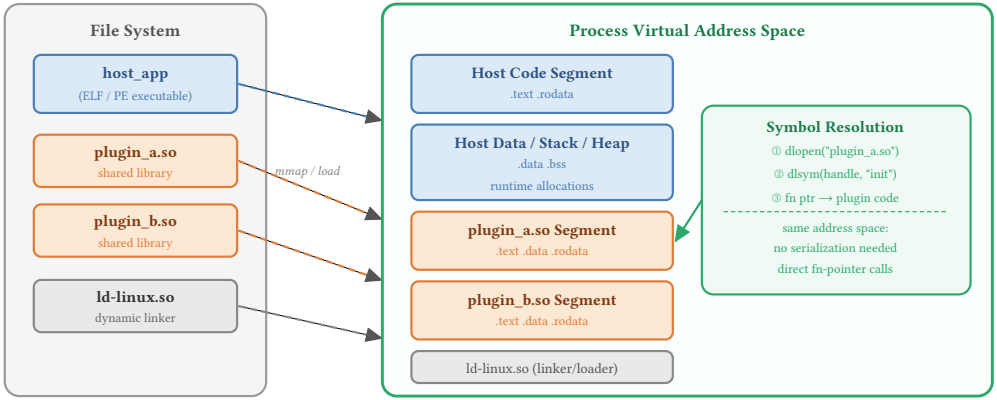
## 2.a.a Application Binary Interface




Related term: **Foreign Function Interface**

## 2.a.b Dynamic Linking/Loading

1) Discovery
2) Loading
3) Resolution
4) Execution



# (Unstable) Rust ABI

## 2.b.a Demonstration of dynamic linking and loading

```rust
// In base library (used by application and plugin):
trait Fuser {
  fn fuse(&self, data: &[SensorData]) -> Result<SensorData, Box<dyn Error>>;
}
```

```rust
// In plugin (library):
struct AveragePlugin;
impl Fuser for AveragePlugin { ... }

#[unsafe(no_mangle)] // ensure name can be found in binary
pub fn average_plugin() -> Box<dyn Fuser> {
    Box::new(AveragePlugin::new())
}
```

# (Unstable) Rust ABI (ii)

```rust
// In application:
type FuseFunc = extern "Rust" fn() -> Box<dyn Fuser>; // use Rust ABI
let plugin: Plugin<FuseFunc> = Plugin::new(
  &plugin_path.into(),
  "average_plugin".as_bytes()
).unwrap(); // 1), 2) and 3)

let fuser: Box<dyn Fuser> = plugin.get();
let sd = [ SensorData::new(24.0, 0.01), ... ];
let res = fuser.fuse(&sd); // 4) Execution
println!("{:?}", res);
```

# (Unstable) Rust ABI (iii)

```rust
// In application:
struct Plugin<T> where T: Fn() -> Box<dyn Fuser> + Copy{
  lib: Library,
  func: T, // or Vec<T> or HashMap<String, T>
}
```

1) Discovery
2) Loading
3) Resolution
4) Execution

Using the "libloading" crate:

```rust
fn new(path: &PathBuf, s: &[u8]) -> Result<Self, libloading::Error> {
  let lib = unsafe { Library::new(path) }?; // 1+2)
  let symbol: Symbol<T> = unsafe { lib.get(s) }?; // 3)
  let func: T = *symbol; // fn deref(&self) -> &T
  Ok(Plugin { lib, func })
}
fn get(&self) -> Box<dyn Fuser> { (self.func)() }
```

# (Unstable) Rust ABI (iv)

## 2.b.b .... but it's Unstable

The implementation itself is not complex, but...

- Compiler version changes **may** impact the generated binary
- Changes to structs, traits and enums **may** impact the generated binary
- Base library, application and plugins need to run on the **same compiler version**

But we can use the **C ABI** (next slides)! But representing Rust in C is very difficult and will introduce trade-offs.

- How are structs, traits and enums represented?
- What happens to VTables?

# abi_stable crate

- Usage of `#[sabi_trait]` attribute macro for creating FFI-safe trait objects
- Allows for a lot of customization when specifying the FFI
- Provides ABI-stable alternatives to many *std-types*, but also some *external_types*
- Widely used crate with a lot of community support

**but**:

- Many hidden structs and types are generated and are required for further use, therefore it is overwhelming at first
- Usage of `async` is not possible, because a FFI-safe wrapper for `Future` is not supported.

# stabby crate

- Easy to get started as primarily `#[stabby::stabby]`, `#[stabby::export]` and `stabby::dynptr!(...)` are used.
- Good explanation of the concepts behind ABI development (and why Rust doesn't like the C ABI and vice versa)
- Provides ABI-stable alternatives to common types
- Built for performance
- "Compiler-change-proof ABI-stability is proven statically through the type system."
- Imported/Exported binaries can be checked for validity

**but**:

- relatively new and therefore not a lot of community engagement (yet)
- had some breaking changes between versions due to a rust compiler update
- documentation can be spotty at times

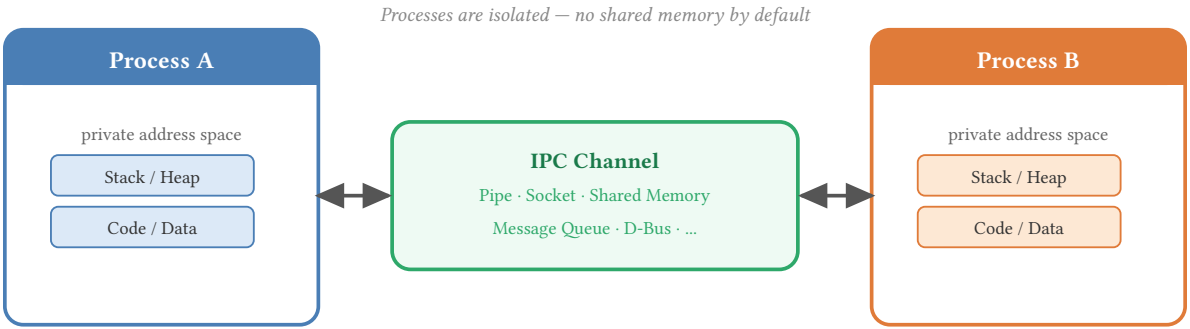
# Comparison ABI crates

| Crate | Perf. Diff | Development Complexity | Limitation | Interoperability |
|---|---|---|---|---|
| **native** | | ++ | 0 | 0 |
| **unstable_abi** | 16% | -- | -- | 0 |
| **abi_stable** | 25% | 0 | 0 | + |
| **stabby** | 9% | 0 | 0 | + |

Table 1: Summary of ABI results.

# 3 IPC

# Interprocess communication




Biggest takeaway: Shared memory is not accessible for this approach. Therefore (de-) **serialization** is required.

# rustbridge crate

High-level JSON, native Rust speed — Work with serde types, not raw pointers!

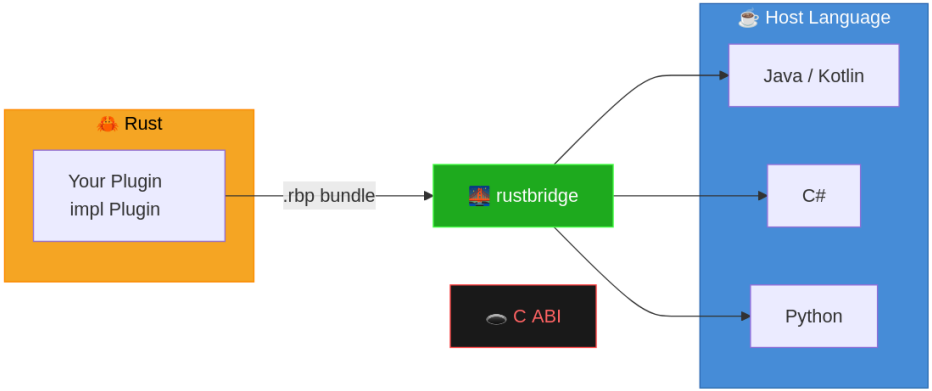



Figure 5: A rustbridge plugin can be bundled into a .rbp file and used across different languages.

# rustbridge crate (ii)

- Stable C ABI — Plugins work regardless of your Rust compiler version or optimization flags
- Managed lifecycle: Startup, shutdown, including logging callbacks
- Excellent technical documentation and examples
- Communication with Request/Response Headers (JSON or Binary)
- Usage of CLI tools to create the bundle

**but**:

- No shared memory, therefore performance loss due to serialization
- Binary approach requires manual conversion of bytes to types
- Requires strict separation of plugin and application

# Comparison rustbridge

| Crate | Perf. Diff | Development Complexity | Limitation | Interoperability |
|---|---|---|---|---|
| **native** | | ++ | 0 | 0 |
| **rustbridge (JSON)** | 537% | + | 0 | ++ |
| **rustbridge (Binary)** | 25% | - | 0 | ++ |

Table 2: Summary of IPC results.

# 4 Conclusion

# Conclusion

| Crate | Perf. Diff | Development Complexity | Limitation | Interoperability |
|---|---|---|---|---|
| **native** | | ++ | 0 | 0 |
| **unstable_abi** | 16% | -- | -- | 0 |
| **abi_stable** | 25% | 0 | 0 | + |
| **stabby** | 9% | 0 | 0 | + |
| **rustbridge (JSON)** | 537% | + | 0 | ++ |
| **rustbridge (Binary)** | 25% | - | 0 | ++ |

Table 3: Summary of results.

# 5 Appendix

# abi_stable implementation

## 5.a.a Base Library

```
#[sabi_trait] // converts this to: Fuser_T0<'lt, Pointer<()>> (and more...)
pub trait Fuser {
  #[sabi(last_prefix_field)]
  fn fuse<'a>(&self, data: RSlice<'a, SensorData>) -> RResult<SensorData, RBoxError>;
}
pub type FuserBox = Fuser_T0<'static, RBox<()>>;
```

```
#[repr(C)]
#[derive(StableAbi)]
#[sabi(kind(Prefix(prefix_ref = FuserLibRef)))]
#[sabi(missing_field(panic))]
pub struct FuserLib{
    pub new_fuser: extern "C" fn() -> FuserBox,
}
```

# abi_stable implementation (ii)

```
impl RootModule for FuserLibRef {
    abi_stable::declare_root_module_statics! {FuserLibRef}
    const BASE_NAME: &'static str = "fuser_lib";
    const NAME: &'static str = "fuser_lib";
    const VERSION_STRINGS: VersionStrings = package_version_strings!();
}
```

```
pub fn load_root_module_in_directory(directory: &Path) -> Result<FuserLibRef,
LibraryError> {
    FuserLibRef::load_from_directory(directory)
}
```

# abi_stable implementation (iii)

## 5.a.b Plugin

```
// in plugin:
#[export_root_module]
fn average_plugin() -> FuserLibRef {
  FuserLib {new_fuser: average_plugin_impl}.leak_into_prefix()
}
#[sabi_extern_fn]
fn average_plugin_impl() -> FuserBox {
  Fuser_TO::from_value(AveragePlugin::new(),
                       TD_Opaque, // disallow DOWNCASTING of trait objects
  )
}
```

# abi_stable implementation (iv)

## 5.a.c Application

```
let path : PathBuf = plugin_path.into();
let data = [ SensorData::new(24.0, 0.01), ... ];
let data = RSlice::from_slice(&data);

let plugin: FuserLibRef = load_root_module_in_directory(&path)?;
let fuser: Fuser_TO<'_, RBox<()>> = plugin.new_fuser()();
let res= plugin.fuse(data);
println!("a) {:?}", res);
```

# stabby implementation

## 5.b.a Base Library

```
#[stabby::stabby(checked)] // neccessary for stabby!
pub trait Fuser {
    extern "C" fn fuse(
        &self,
        data: stabby::slice::Slice<'_, SensorData>,
    ) -> stabby::result::Result<SensorData, error::Error>;
    extern "C" fn print(&self);
}
```

# stabby implementation (ii)

## 5.b.b Plugin

```
#[stabby::stabby]
pub struct AveragePlugin { }
impl Fuser for AveragePlugin { ... }

#[stabby::export]
pub extern "C" fn average_plugin()
  -> stabby::dynptr!(Box<dyn Fuser>) {
  Box::new(AveragePlugin::new()).into()
}
```

# stabby implementation (iii)

## 5.b.c Application

```
pub type FuseFunc = extern "C" fn() -> FuserPluginDyn;
pub type FuserPluginDyn = stabby::dynptr!(stabby::boxed::Box<dyn
app_core::fuse::Fuser>);

pub struct Plugin<TDyn> {
  // Keep the plugin value before the library handle so it is dropped first.
  func: TDyn,
  _lib: Library,
}

impl<TDyn> Plugin<TDyn> {
    pub fn new<TFunc>(
        plugin_name: &PathBuf,
        symbol: &[u8],
    ) -> Result<Self, Box<dyn Error + Send + Sync>>
```

# stabby implementation (iv)

```
where
    TFunc: PluginFactory<Dyn = TDyn>, // allows to use of  "get(&self)"
{
    let lib = unsafe { libloading::Library::new(plugin_name)? };
    let my_imported_function = unsafe { lib.get_stabbied::<TFunc>(symbol)? };
    let fund = *my_imported_function;
    let tdyn = fund.get();

    Ok(Self {
        func: tdyn,
        _lib: lib,
    })
}

pub fn get(&self) -> &TDyn {
    &self.func
}
```

# stabby implementation (v)

```
    pub fn get_mut(&mut self) -> &mut TDyn {
        &mut self.func
    }
}

pub fn get_plugin<TFunc>(
    plugin_name: PathBuf,
    symbol: &[u8],
) -> Result<Plugin<TFunc::Dyn>, Box<dyn Error + Send + Sync>>
where
    TFunc: PluginFactory,
{
    Plugin::<TFunc::Dyn>::new::<TFunc>(&plugin_name, symbol)
}

let mut plugin: Plugin<FuserPluginDyn> = get_plugin::<FuseFunc>(plugin_path.into(),
```

# stabby implementation (vi)

```
"average_plugin".as_bytes()).unwrap();
let data = [ SensorData::new(24.0, 0.01), ... ];
let data: Slice<'_, SensorData> = Slice::new(&data);
let fuser = plugin.get();
let res = fuser.fuse(data);
println!("a) {:?}", res);
```

# rustbridge implementation

## 5.c.a Plugin (JSON)

```rust
#[derive(Debug, Clone, Serialize, Deserialize, Message)]
#[message(tag = "fuse")]
pub struct FuseRequest {
    pub sd: Vec<SensorData>,
}

/// Fuse response message
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct FuseResponse {
    pub sd: SensorData,
}

#[derive(Default)]
pub struct FuserPlugin;
```

# rustbridge implementation (ii)

```
impl FuserPlugin {
    pub fn new() -> Self {
        Self
    }

    fn handle_fuse(&self, req: FuseRequest) -> PluginResult<FuseResponse> {
        tracing::info!("Received fuse request: {:?}", req);
        let fused_data = average_fuse(&req.sd);
        Ok(FuseResponse { sd: fused_data })
    }
}

#[async_trait]
impl Plugin for FuserPlugin {
    async fn on_start(&self, _ctx: &PluginContext) -> PluginResult<()> {
        tracing::info!("fuser-plugin starting...");
        register_binary_handlers();
```

# rustbridge implementation (iii)

```
        tracing::info!("fuser-plugin sucessfully started");
        Ok(())
    }

    async fn handle_request(
        &self,
        _ctx: &PluginContext,
        type_tag: &str,
        payload: &[u8],
    ) -> PluginResult<Vec<u8>> {
        match type_tag {
            "fuse" => {
                let req: FuseRequest = serde_json::from_slice(payload)?;
                let resp = self.handle_fuse(req)?;
                Ok(serde_json::to_vec(&resp)?)
            }
            _ => Err(PluginError::UnknownMessageType(type_tag.to_string())),
```

# rustbridge implementation (iv)

```
        }
    }

    async fn on_stop(&self, _ctx: &PluginContext) -> PluginResult<()> {
        tracing::info!("fuser-plugin stopped");
        Ok(())
    }
}
```

### 5.c.b Application (JSON)

### 5.c.c Plugin (Binary)

```
/// Message ID for fuser creation
pub const MSG_FUSER_CREATE: u32 = 100;
```

# rustbridge implementation (v)

```rust
/// Request header for fuser creation (24 bytes)
///
/// Layout:
///   Offset 0:  version (1 byte)
///   Offset 1:  _reserved (3 bytes, padding for alignment)
///   Offset 4: payload_size (4 bytes, u32: size of sensor data following header)
///   Total: 8 bytes
#[repr(C)]
#[derive(Debug, Clone, Copy)]
pub struct FuserRequestHeader {
    /// Struct version for forward compatibility
    pub version: u8,
    /// Reserved for alignment
    pub _reserved: [u8; 3],
    // Size of sensor data following this header
    pub payload_size: u32, // NOTE: NO POINTERS
}
```

# rustbridge implementation (vi)

```rust
impl FuserRequestHeader {
    pub const VERSION: u8 = 1;
    pub const SIZE: usize = 8;
}

/// Response header for fuser creation (8 bytes)
#[repr(C)]
#[derive(Debug, Clone, Copy)]
pub struct FuserResponseHeader {
    pub version: u8,
    pub _reserved: [u8; 3],
    pub payload_size: u32,
}

impl FuserResponseHeader {
    pub const VERSION: u8 = 1;
```

# rustbridge implementation (vii)

```
    pub const SIZE: usize = 8;
}
```

## 5.c.d Application (Binary)